16
2×2×2×2
11 22
1 3 5 7, 9, 11, 13

- Prime P.

$Z_p = \{0, 1, 2, \ldots, p-1\}$

$(Z_5, +_5, *_5) \rightarrow$ Field.

- 2 is a generator.
  3 is also a generator

- Unkown x, generator a g, Modulus p ($g^x \bmod p$) (value given)
  ↳ find x in a polynomial time of $\log p$.

★ RSA pseudorandom bit generator:-

★ SUMMARY:-

A pseudorandom bit sequence $z_1 z_2, \ldots, z_l$ of length l is generated.

1) Setup:- Generate 2 secret RSA like primes p & q and compute $n = pq$ and $\phi(n) = (p-1)(q-1)$.

2.) Select a random integer e, s.t. $1 < e < \phi$, $\gcd(e, \phi) = 1$.

3) Select a random integer x_0 (the seed) in the interval $[1, n-1]$

4.) For i from 1 to l do:

a) $x_i \leftarrow x_{i-1}^{e} \bmod n$.

(b) $z_i \leftarrow$ the least significant bit of x_i

5.) output the sequence $z_1, z_2, z_3, \ldots, z_l$.

* Indistinguishability of Probability Distribution:-

· 2 Properties of PRBG:-

(i) It should be fast.

(ii) It should be secure.

↓

A string of l bits produced by PRBG should look random.

That is it should be impossible in an amount of time that is polynomial in k (equivalently polynomial in l) to distinguish a string of l bits produced by PRBG from a string of l truely random bits.

* Let a bit generator produces "1"s with prob. $\frac{2}{3}$ then, it should be easy to distinguish the bit stream from a truely random bit stream.

* Distinguishing strategy:-

Given a bit string of length l denote the number of 1's in the bitstring by l_1.

- On an average a random bit string of length l will contain $l/2$ 1's.
- Our generator's bit strings expected number of 1's is $\frac{2l}{3}$.
- Let $l_{average} = \frac{l/2 + 2l/3}{2} = \frac{7l}{12}$

If $l_1 > \frac{7l}{12}$ ($= l_{average}$) we would ~~give~~ guess the bit string was more likely to be generated from the biased generator.

- $z^i = (z_1, z_2, \cdots, z_i)$ i-tuple.
- Def^n:-

Suppose p_0 and p_1 are 2 probability distribution on ~~z to l~~. $(z_2)^l$ of all bit strings of length l.

- For $j = 0, 1$ and $z^l \in (z_2)^l$
- $p_j(z^l)$ denote the prob. of the string z^l occuring in the distribution p_j.

- Let $dst : (Z_2)^l \to \{0,1\}$ be a function and let $\epsilon > 0$. For $j = 0, 1$ ~~a function~~ define

$$E_{dst}(p_j) = \sum_{\{z^l \in (Z_2)^l \Rightarrow dst(z^l) = 1\}} p_j(z^l)$$

We say a distinguisher is ϵ-distinguishable of p_0 and p_1 provided

$$|E_{dst}(p_0) - E_{dst}(p_1)| \geq \epsilon$$

and we say that p_0 and p_1 are ϵ-distinguishable if there is an ϵ-distinguisher of p_0 and p_1.

- Given l-tuple $z^l = (z_1, z_2, \ldots, z_l)$ a distinguisher might ~~give~~ guess 0 with some probability p (depending on z^l) and hence guess 1 with probability $1-p$)

- In the case of randomized distinguisher

$$E_{dist}(p_j) = \sum_{z^l \in (Z_2)^l} p_j(z^l) \times Pr[dist(z^l) = 1]$$

* Relevance of distinguishers to PRBG:-

Consider the seq. of l bits produced by a bit generator

- There are 2^l possible sequences and if l bits were chosen independently and uniformly at random, then each of the 2^l seq. would occur with prob. $\frac{1}{2^l}$.

- Let this uniform prob. distribution be p_u. Now consider sequences produced by bit generator f: suppose a k-bit seed is choosen uniformly at random and generator computes bit string of length l

- Then we obtain a prob. distribution on the set of all bit strings of length l and denote it by p_f.

- For 2^l possible sequences 2^k sequences each occur with prob. $\frac{1}{2^k}$ and remaining $2^l - 2^k$ sequences never occur ∴ Hence prob. distribution p_f is very non-uniform.

Eg:- Suppose a bit generator f only produces sequences in which exactly $l/2$ bits have the value 0 and $\frac{l}{2}$ bits have value 1. Define:

$$\text{dist.}(z_1, z_2, \ldots, z_l) = \begin{cases} 1 & \text{if } (z_1, z_2, \ldots, z_l) \text{ have } \frac{l}{2} \text{ bits of 0.} \\ 0 & \text{otherwise} \end{cases}$$

$$E_{\text{dst}}\, p_u = \binom{l}{l/2} \frac{1}{2^l}$$

$Edist(p_f) = 1$

Let $t \to \infty$ $\quad \frac{\binom{t}{t/2}}{2^t} = 0$

For any fixed value of $\epsilon < 1$ p_u and p_f are

ϵ- distinguishable.

* **Next Bit Predictor:-**

Let f be a (k, l) bit generator suppose $1 \le i \le l-1$, we have the function:*

$$\underset{\text{next bit predictor}}{nbp} : (Z_2)^{i-1} \to Z_2$$

which takes input a $(i-1)$ tuple $z^{i-1} = z_1, z_2, \ldots z_{i-1}$.

and predicts the next i^{th} bit.

* is an ϵ bit predictor for f if and only if

$$\sum_{z^{i-1} \in (Z_2)^{i-1}} (p_f(z^{i-1}) \times Pr[z_i = nbp(z^{i-1}) | z^{i-1}] \ge \frac{1}{2} + \epsilon$$

* **Pf:** The probability of correctly predicting the i^{th} generated bit is computed by summing over the all possible $(i-1)$ tuples $z^{i-1} = z_1, z_2, \ldots, z_{i-1}$ the product of the probability that the $(i-1)^{th}$ tuple z^{i-1} is produced by the bit generator f and the probability that the i^{th} bit

is predicted correctly, given $(i-1)$ tuple z^{i-1}

* Linear Congruential generator:-

Suppose $M \geq 2$ is an integer and suppose $1 \leq a, b \leq M-1$.

- Define $k = 1 + \lfloor \log_2 M \rfloor$ and let $k+1 \leq l \leq M-1$
- The seed is an integer so where $0 \leq s_0 \leq M-1$
- (Observe that the binary representation of a seed is a bit string of length not exceeding k, however not all bit string of length k are permissible).

$s_i = (a s_{i-1} + b) \bmod M$
for $1 \leq i \leq t$ and then define:
$f(s_0) = z_1 z_2, \ldots, z_t$

- where $z_i = s_i \bmod 2$
$1 \leq i \leq l$. Then f is a (k, l) Linear congruential generator.

- Let us take $a = 3, b = 5, M = 31$
$s_i = (3 s_{i-1} + 5) \bmod 31$

- $nbp(z^{i-1}) = 1 - z_{i-1}$

DISTINGUISH (z^i)
external nbp
if $z = z_i$ then return ①
else return ⓪

For Z_{31} for $s_{i-1} = 13$

$s_i = 13$

other remainders form 30-cycle

our nbp is a $\frac{20}{31}$ distinguisher

$\downarrow$

$\left(\frac{1}{2} + \frac{9}{62}\right)$ [verify yourself] ★

★ Blum-Blum-Shub-Generator:-

- Quadratic Residues:-

$$QR(n) = \{ x^2 \bmod n : x \in Z_n^* \}$$

Algorithm:-

Let p, q be $2\ (k/2)$ bit primes s.t.

$$p \equiv q \equiv 3 \bmod 4.$$

- Define $n = p \cdot q$. Let $QR(n)$ denote the set of quadratic residues modulus n.

- A seed s_0 is any element of $QR(n)$.
For $0 \le i \le l-1$
define $s_{i+1} = s_i^2 \bmod n$ and then define
$f(s_0) = (z_1, z_2, \dots, z_l)$
where $z_i = s_i \bmod 2 \quad 1 \le i \le l$

- Then f is a (k, l) PRBG called Blum Blum Shub generator which we abbreviate as BBS generator.

- one way to choose an appropriate seed is to select an element $s-1 \in Z_n^*$ and compute $s_0 = (s-1)^2 \bmod n$
$\Rightarrow s_0 \in QR(n)$.

$$z_i = (s_0^{2^i} \bmod n) \bmod 2.$$

* Jacobi Symbol:-

Suppose p and q are 2 distinct odd primes and let $n = p \cdot q$.

Jacobi symbol $\rightarrow$ $\left(\frac{x}{n}\right) = \begin{cases} 0 & \text{if } \gcd(x, n) > 1 \\ 1 & \text{if } \left(\frac{x}{p}\right) = \left(\frac{x}{q}\right) = 1 \text{ or } -1 \\ -1 & \text{if one of } \left(\frac{x}{p}\right), \left(\frac{x}{q}\right) = +1 \\ & \text{on other} = -1 \end{cases}$

- Def^n:-

Suppose n is an odd +ve integer and the prime factorization of n is $n = \prod_{i=1}^{k} p_i^{e_i}$

Let a be an integer. The Jacobi symbol

$$\left(\frac{a}{n}\right) = \prod_{i=1}^{k} \left(\frac{a}{p}\right)^{e_i}$$

x is a quadratic residue modulo n iff

$$\left(\frac{x}{p}\right) = \left(\frac{x}{q}\right) = +1$$

Define

$$\widetilde{QR}(n) = \{ x \in Z_n^* \mid QR(n), \left(\frac{x}{n}\right) = 1 \}$$

Pseudo square modulo n

$$\widetilde{QR}(n) = \{ x \in Z_n^* : \left(\frac{x}{p}\right) = \left(\frac{x}{q}\right) = -1 \}$$

$$\# |QR(n)| = |\widetilde{QR}(n)| = \frac{1}{4} \cdot \frac{(p-1)(q-1)}{4}$$

* Composite Quadratic Residues:-

- Instance:- A +ve integer n that is the product of 2 unknown distinct odd primes p and q and an integer $x \in Z_n^*$ such that $\left(\frac{x}{n}\right) = 1$.

Ques:- Does $x \in QR(n)$?

Composite Quadratic residues problem require us to distinguish quadratic residues mod n from pseudo squares mod n. This can be no more difficult than factoring n.

- There does not seem to be anyway to solve composite quadratic residues efficiently if the factorization of n is not known. so it is commonly conjectured that this problem is intractable if it is infeasible to factor n.

- In BBS generating sequence for any quadratic residue x there is a unique square root of x that is also a quadratic residue.
- This particular square root is called principal square root.
- As a consequence of the mapping $x \mapsto x^2 \bmod n$ which is used to define BBS generator, is a permutation on $QR(n)$ the set of quadratic residues modulo n.

Ex:- Suppose $n = 253 = 11 \times 23$ Then

$$|QR(n)| = \frac{10 \times 22}{4} = 55$$

- It can be shown that BBS generator in Z_{55} permutes the elements of $|QR(55)|$ in one cycle of length-1, one cycle of length-4, one cycle of length-10 and 2 cycles of length 20.

* Security of BBS Generator:-

A previous bit predictor for a (k, l) BBS generator will take as input l pseudo random bits produced by the generator (as determined by an unknown random seed $s_0 \in QR(n)$) and attempt to compute $z_0 = s_0 \bmod 2$

p.b.p is an ϵ-previous bit predictor if the prob. of

guessing z_0 correctly is $\frac{1}{2}+\epsilon$.

Theorem:- Suppose $\exists$ a polynomial time ϵ-distinguisher of p_f and p_u where p_f is the prob. distribution induced on $(Z_2)^l$ by (k,l) B.B.S generator f and p_u is the uniform prob. distribution on $(Z_2)^l$. Then $\exists$ a polynomial time $\frac{\epsilon}{l}$ previous bit predictor for f.

QR-Test (x,n):-
external p.b.p.

$s_1 \leftarrow x^2 \bmod n$

$z_1 \leftarrow s_1 \bmod 2$

use B.B.S generator to compute $z_2, z_3, \ldots, z_l$ from seed s_1

$z \leftarrow$ p.b.p$(z_1, z_2, \ldots, z_l)$

If $(x \bmod 2) = z$ then return ('YES')
else return (NO).

Theorem:- Suppose pbp is a (poly^n time) $\frac{\delta}{l}$-previous bit predictor for (k,l)-BBS Generator f. Then the algorithm QR-Test determines quadratic residues correctly in polynomial time with prob. at least $\frac{1}{2}+\delta$ where the probability

is averaged over all possible inputs $x \in QR(n) \cup \overline{QR(n)}$ which are chosen uniformly at random.

* Monte Carlo (MC):-

MC-QR-Test (x)

external QR-Test

Choose $r \in Z_n^*$ randomly

$x' \leftarrow r^2 x \bmod n$

suppose $s \in \{1, -1\}$ randomly $x' \leftarrow s x' \bmod n$

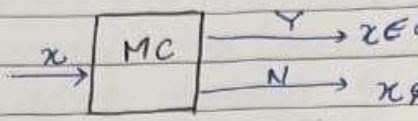
$t \leftarrow$ QR-Test (x')

if ((t = Yes) and (s=1)) or ((t= No) and s=-1))
then return (YES)
else return (NO).

Theorem:- Suppose that QR-Test determines the quadratic residuesity correctly (in polynomial time) with average prob. at least $\frac{1}{2} + \delta$.

Then the algorithm MC-QR-Test is a polynomial time Monte-Carlo algorithm for composite Quadratic Residues.

having error probability at $\frac{1}{2} - \delta$.



Theorem:- Suppose A is an unbiased ~~with~~ Monte Carlo algorithm with error probability at most $\frac{1}{2} - \delta$.

Suppose we define an algorithm A^n by running A, $n = 2m+1$ times on ar given instance I and outputting the most frequent answer. Then the error probability of algorithm A^n is at most

$$\frac{(1 - 4\delta^2)^m}{2}$$

Proof:- The probability of abtaining i correct answers in n trials is at most:

$$\binom{n}{i}\left(\frac{1}{2}+\delta\right)^i\left(\frac{1}{2}-\delta\right)^{n-i}$$

The probability that most frequently occurring answer is incorrect = the prob. that the number of correct answers in n trials is atmost m.

$$P[\text{error}] \leq \sum_{i=0}^{m} \binom{n}{i}\left(\frac{1}{2}+\delta\right)^i\left(\frac{1}{2}-\delta\right)^{n-i}$$

$$= \left(\frac{1}{2}+\delta\right)^m\left(\frac{1}{2}-\delta\right)^{m+1} \sum_{i=0}^{m}\binom{n}{i}\frac{\left(\frac{1}{2}-\delta\right)^{m-i}}{\left(\frac{1}{2}+\delta\right)^{m-i}}$$

$$\leq \left(\frac{1}{2}+\delta\right)^m \left(\frac{1}{2}-\delta\right)^{m+1} \sum_{i=0}^{m} \binom{n}{i}$$

$$\sum_{i=0}^{m} \binom{n}{i} = \frac{1}{2} \times 2^{2m+1} = 2^{2m}$$

$$P[\text{error}] \leq \left(\frac{1}{4}-\delta^2\right)^m \left(\frac{1}{2}-\delta\right) 2^{2m}$$

$$\leq (1-4\delta^2)^m \left(\frac{1}{2}-\delta\right) \leq \frac{(1-4\delta^2)^m}{2}$$

Suppose we want to lower the error prob. to some value γ.

where $0 < \gamma < \frac{1}{2} - \delta$.

we need to choose m so that $\frac{(1-4\delta^2)^m}{2} \leq \delta$.

$$m \log(1-4\delta^2) - 1 \leq \log \gamma$$

$$m = \left\lceil \frac{\log \gamma + 1}{\log(1-4\delta^2)} \right\rceil$$

Example:- Suppose we start with a monte carlo algo. that gives correct answer with probability at least 0.55 so $\delta = 0.05$. If we desire a MC algo. in which prob. of error is at most 0.05.

then it suffices to take $m = 230$,
$n = 461$.

- (k,l) - BBS generator can be ϵ-distinguished from l random bits $\Rightarrow$ $\left(\frac{\epsilon}{l}\right)$ previous bit predictor for (k,l)-BBS generator $\Rightarrow$ Distinguishing algorithm for composite quadratic residues that is correct with pro. at $\frac{1}{2}+\frac{\epsilon}{l}$

$\Rightarrow$ Unbiased Monte Carlo Algorithm for Composite Quadratic ~~Reduces~~ residues having error prob. at most $\frac{1}{2}-\frac{\epsilon}{l}$

$\Rightarrow$ Unbiased MC algo. for Composite Quadratic residues having error prob. at most γ, for any $\gamma > 0$.

* Efficiency:-

The sequence of PRB is constructed by taking the lsb of each s_i where $s_i = s_{i-1}^2 \bmod n$. Suppose instead we extracted r lsb bits of each s_i for some +ve integer r and we need to ask if PRB will remain secure (assuming intractability of composite quadratic residues).

- It has been shown that this approach is secure provided $r \leq \log\log n$. so we can extract $\log\log n$ pseudo random bits for modular sequering.

- In realistic implementation BBS generator, say one in which $n \approx 10^{160}$, we can extract 9 bits per squaring operation.

Linear Feedback Shift Register :- (LFSR)

- Random Number Generator
- Stream Cipher → Here speed is more important than security.

- Suppose we have a sequence 0100001001011..... can be described by giving the initial sequence initial values:

$x_1 \equiv 0, x_2 \equiv 1, x_3 \equiv 0, x_4 \equiv 0, x_5 \equiv 0,$

- Linear Reccurrence Relation:-

$$x_{n+5} \equiv x_n + x_{n+2} \pmod 2$$
$$x_6 \equiv x_1 + x_3 = 0$$
$$x_7 \equiv x_2 + x_4 = 1$$

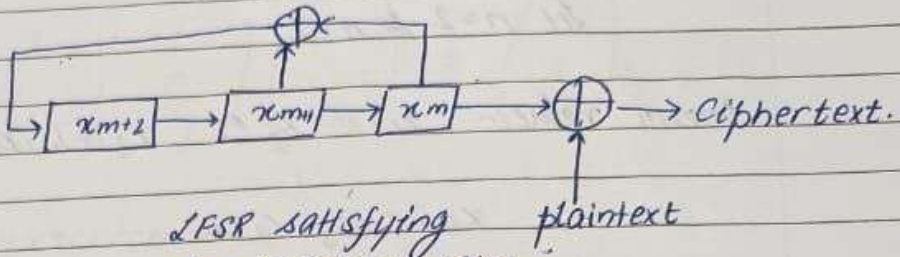
- More generally:-

$$x_{n+m} = C_0 x_n + C_1 x_{n-1} + \cdots + C_{m-1} x_{m-1} \pmod 2$$

- Plaintext:- 1011001------
- Key:- 0100001
- Ciphertext:- 1111000
- Plaintext = Cipher + Key = 1011001

__/__/__

- One Advantage:- is that key with a large period can be generated using very little information.



- For each increment of a counter the bit in each box is shifted to other boxes as indicated. The ⊕ symbol denoting addition mod 2 of the incoming bit.
- The output which is taken x_m is added to the next bit of plaintext to produce ciphertext.
- Here, LFSR represents the recurrence:
$$x_{m+3} \equiv x_{m+1} + x_m \pmod 2$$
- once the initial values of x_1, x_2 and x_3 are specified the m/c produces bits very efficiently.
- Goal is: to use the portion of the key to deduce the coefficients of the recurrence and ~~con~~ consequently compute all the rest of the key.
- Initial Fragment and Keystream:

01101011100

start with length 2 recurrence:-

$$x_{n+2} = c_0 x_n + c_1 x_{n+1}$$

Let $n=2$ & $n=2$

Known value:- $x_1 = 0$ $x_2 = 1$, $x_3 = 1$, $x_4 = 0$

$$x_5 = c_0 x_3 + c_1 x_4$$

~~$1 = c_0 + c_1$~~

$1 = c_0 \cdot 0 + c_1 \cdot 1$ ---(1)

$n = 2$

$x_4 = c_0 \cdot x_2 + c_1 x_3$

~~$0 = [1\ 0]$~~ $0 = c_0 \cdot 1 + c_1 \cdot 1$ (2)

* Matrix form $\begin{pmatrix} 0 & 1 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} c_0 \\ c_1 \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$

sol^n $c_0 = c_1 = 1$

$$x_{n+2} = x_n + x_{n+1}$$

- Unfortunately: $x_6 \neq x_4 + x_5$.

∴ We try with ~~an~~ length 3.

- Part of Key string:- 0 1 1 0 1 0 1 1 1 1 0 0

$$x_{n+3} = c_0 x_n + c_1 x_{n+1} + c_2 x_{n+2}$$

$x_4 = 0 = c_0 = c_0 \cdot 0 + c_1 \cdot 1 + c_2 \cdot 1$

$x_5 = 1 = c_0 \cdot 1 + c_1 \cdot 1 + c_2 \cdot 0$

$x_4 = 0 = c_0 \cdot 1 + c_1 \cdot 0 + c_2 \cdot 1$

Matrix form:-

$$\begin{pmatrix} 0 & 1 & 1 \\ 1 & 1 & 0 \\ 1 & 0 & 1 \end{pmatrix} \begin{pmatrix} c_0 \\ c_1 \\ c_2 \end{pmatrix} = \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix}$$

- Resulting recurrence:-
  $x_{n+1} \equiv x_n + x_{n+1}$

In general to test for a recurrence of length m we assume $x_1, x_2, \cdots, x_m$

$$\begin{pmatrix} x_1 & x_2 & \cdots & x_m \\ \vdots & & & x_{m+1} \\ & & & \vdots \\ x_m & \cdots & & x_{2m} \end{pmatrix} \begin{pmatrix} c_0 \\ c_1 \\ c_2 \\ \vdots \\ c_{m+1} \end{pmatrix} = \begin{pmatrix} 1 \\ \vdots \end{pmatrix} \cdots$$

- A strategy for finding the coefficients of the recurrence:
- Suppose we know 1st 100 bits of the key. For $m = 2, 3, 4, \cdots$ form $m \times m$ matrix and compute its determinant.
- If several consecutive values of m yield 0 determinants, stop. The last m bits to yield non-zero determinant (i.e. 1 mod 2)
- is probably the length of recurrence.

Example:- Seq. of bits:
1001100100111000110000100 0 ---

$$x_{n+8} = x_n + x_{n+1} + x_{n+4}$$

- Suppose we know from the middle of the sequence.
  $x_{17} = 1$, $x_{18} = 0$, $x_{19} = 0$, - - -

Write the recurrence as:

$$x_n \equiv x_{n+1} + x_{n+4} + x_{n+8}$$

$\because$ We are working in mod 2 $\quad -x_n = x_n$

$$x_{16} = x_{17} + x_{20} + x_{24}$$
$$= 1 + 0 + 1 = 0.$$

On continuing we can get $x_{15}, - - , x_1$.

★ Proposition:-

Let $x_1, x_2, - -,$ be the seq. of bits produced by a linear recurrence mod 2.

For each $n \geq 1$ Blet

$$M_n = \begin{pmatrix} x_1 & x_2 & - - & x_n \\ x_2 & x_3 & & x_{n+1} \\ | & | & & | \\ x_n & x_{n+1} & & x_{2n-1} \end{pmatrix}$$

Let N be the length of the shortest recurrence that generate the seq. $x_1, x_2, - -,$ Then

$$\det(M_N) \equiv 1 \pmod 2 \text{ and}$$
$$\det(M_n) \equiv 0 \pmod 2 \ \forall\ n > N.$$

Proof:-

Note:- Length 3 recurrence

$$x_{n+3} \equiv x_{n+2}$$

↓ shorter relation

$$x_{n+1} \equiv x_n \ (\forall n \geq 2)$$

- seq. :- 1, 0, 1, 1, 0, 1, 1, 0, 1, 1, 0, 1

$$x_{n+4} = x_{n+3} + x_{n+1} + x_n$$

- A shorter recurrence :

$$x_{n+2} = x_{n+1} + x_n$$

∴ If there is a recurrence of length N and if $n > N_1$, then one row of Matrix M_n is congruent mod 2 ~~wit~~ to a linear combination of other row.

- Suppose $\det(M_N) \equiv 0 \pmod 2$
  ⇒ ∃ a non zero row vector $\bar{b} = (b_0, b_1, \dots, b_{N-1})$
  s.t. $\bar{b} \cdot M_n \equiv 0$

- We will show that this gives a recurrence relation for the seq. $x_1, x_2, \dots, ?$ etc. and that the length of this relation is less than N. This ~~contradicts~~ contradicts the assumption that N is smallest.

- The contradiction implies $\det(M_n) \equiv 1 \pmod 2$

Let the recurrence of length N be:-

$$x_{N+n} = c_0 x_n + c_1 x_{n+1} + \dots + c_{N-1} x_{N-1+n}$$

- For each $i \geq 0$, let

$$M^{(i)} = \begin{pmatrix} x_{i+1} & x_{i+2} & \cdots & x_{i+N} \\ \vdots & \vdots & & \vdots \\ x_{i+N} & x_{i+N+1} & \cdots & x_{i+2N-1} \end{pmatrix}$$

- Then, $M^{(0)} = M_N$. The recurrence relation implies that

$$M^{(i)} \begin{pmatrix} c_0 \\ c_1 \\ \vdots \\ c_{N-1} \end{pmatrix} = \begin{pmatrix} x_{i+N+1} \\ \vdots \\ x_{i+2N} \end{pmatrix}$$

which is the last column of $M^{(i+1)}$

- By the choice of $\bar{b}$, we have $\bar{b} \cdot M^{(0)} = 0$. Suppose that we know $\bar{b} \cdot M^{(i)} = 0$ for some i. Then

$$\bar{b} \cdot \begin{pmatrix} x_{i+N+1} \\ \vdots \\ x_{i+2N} \end{pmatrix} = \bar{b} \cdot M^{(i)} \cdot \begin{pmatrix} c_0 \\ c_1 \\ c_2 \\ c_3 \end{pmatrix} \equiv 0 \pmod 2.$$

- Therefore, $\bar{b}$ annihilates the last column of $M^{(i+1)}$. Since, the remaining columns of $M^{(i+1)}$ are columns of $M^{(i)}$; we find that $\bar{b} M^{(i+1)} \equiv 0$. By induction we obtain

$$\bar{b} M_0^{(i)} \equiv 0 \quad \forall i \geq 0.$$

- Let $n \geq 1$. Then first column of $M^{(n-1)}$ yields

$$b_0 x_n + b_1 x_{n+1} + \cdots + b_{N-1} x_{n+N-1} \equiv 0$$

- since $\bar{b}$ is not a zero vector $b_j \neq 0$ for at least one j.
- Let m be the largest j, s.t. $b_j \neq 0$, which means $b_m = 1$
  $b_m = 1 \cdot b_m x_{n+m-1} = -x_{n+m-1}$.
- $\therefore$ we can rearrange the relation $x_{n+m} = b_0 x_n + b_1 x_{n+1} + \cdots + b_{n-1} x_{n+m-2}$

This is a recurrence of length $m-1$. Since $m-1 < N$ and N is assumed to be the shortest possible length, we have a contradiction.

$\therefore$ our assumption $\det(M_N) \equiv 0$ must be false. and so $\det(M_N) = 1$

- LFSR:-

Suppose the length of the recurrence is m. Any m consecutive terms of the sequence determine all future elements and by reversing the recurrence all previous values too.

- Clearly if we have m consecutive 0's then all future and previous values are 0's. Therefor, we exclude the case from consideration.
- There are $2^m - 1$ strings of 0's and 1's of length m in which atleast one term is non-zero.

- ∴ As soon as there are more than $2^m - 1$ terms, some string of length m, must occur twice so the sequence repeats.
- The period of the sequence is at most $2^m - 1$

Associated to the recurrence.

$$x_{n+m} \equiv c_0 x_n + c_1 x_{n+1} + \cdots + c_{m-1} x_{n+m-1} \pmod 2$$

there is a polynomial

$$f(T) = T^m - c_{m-1} T^{m-1} - \cdots - c_0$$

- if f(T) is irreducible (cannot factor) ~~is irreducible~~ then it can be shown that the period divides $2^m - 1$. An interesting case when $2^m - 1 \rightarrow$ prime (Mersenne Prime).
- The period is maximal; namely $2^m - 1$.

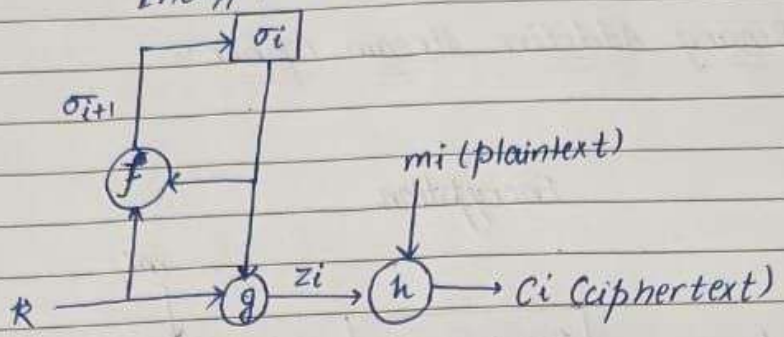
★ String Ciphers:-

1) Synchronous stream Cipher:-

Here keystream is generated independently of plaintext and ciphertext.

__/__/__

Encryption



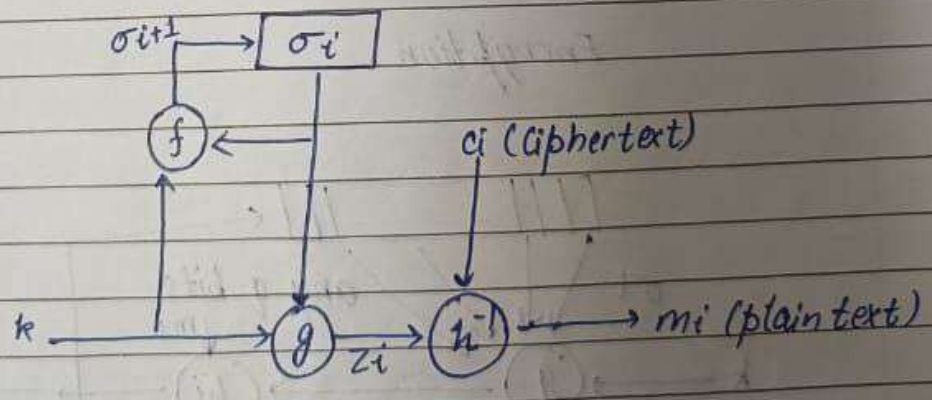
Key: k Keystream: z_i State: σ_i

$$\sigma_{i+1} = f(\sigma_i, k)$$

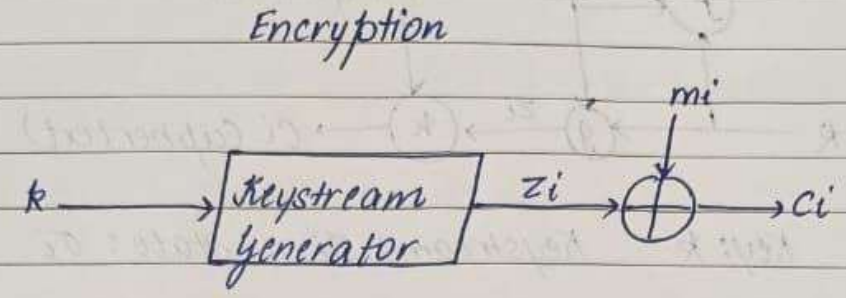
$$z_i = g(\sigma_i, k)$$

$$c_i = h(z_i, m_i)$$

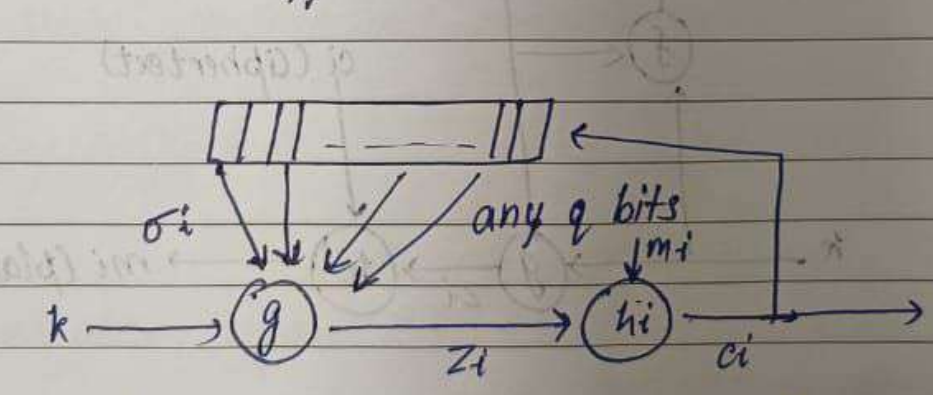
Decryption



2.) Binary Additive stream cipher:-



3.) Self synchronizing stream cipher:-

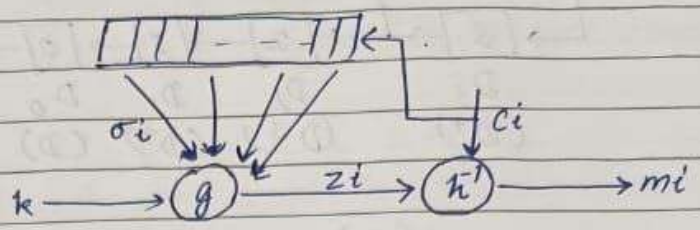


$\sigma_i = (c_{i-t}, c_{i-t+1} \text{ --- }, c_{i-1})$

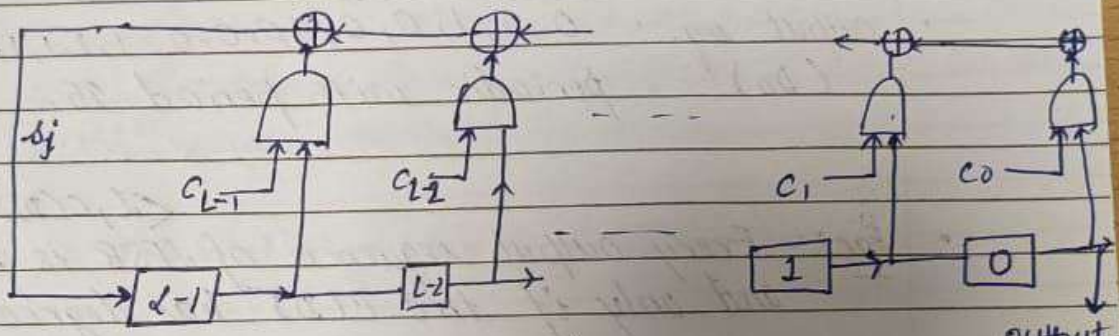
$z_i = g(\sigma_i, k)$

$c_i = h(z_i, m_i)$

Decryption:-



- **LFSR:-**



LFSR of length L. $\langle L, C(D) \rangle$ where

$$C(D) = 1 + C_1 D + C_2 D_2^2 + \cdots + C_L D^L$$

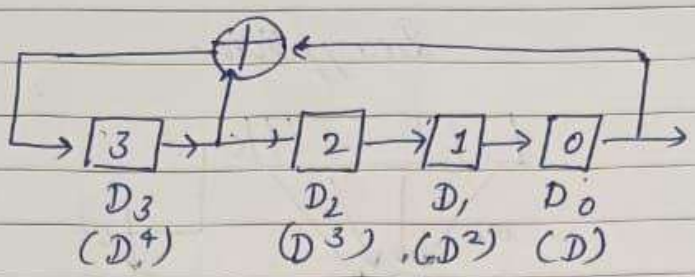
↑ connection polynomial

- LFSR is said to be non-singular if degree of C(D) is L (i.e., $C_L = 1$)
- Initial state of LFSR is $[s_{L-1}, \cdots, s_1, s_0]$, then the output seq:

$$s = s_0, s_1, s_2, \cdots$$

where,

$$s_j = (C_1 s_{j-1} + C_2 s_{j-2} + \cdots + C_L s_{j-L}) \bmod 2 \quad j > L$$



$$C(D) = 1 + D + D^4$$

$$LFSR = \langle 4, 1 + D + D^4 \rangle$$

- output seq. : (D_0) 0, 1, 1, 0, 0, 1, 0, 0, 0, 1, 1, 1, 1, 0, 1 - - - periodic with period 15.

- Fact:- Every output sequence of LFSR is periodic if and only if the $C(D)$ has degree L. $\langle L, C(D) \rangle$

- Fact:- Let $C(D) \in Z_2(D)$ be a connection polynomial of degree L.

(i) If $C(D)$ is irreducible over Z_2 then each of the $2^L - 1$ non-zero initial states of the non-singular LFSR. $\langle L, C(D) \rangle$ produces an output seq. with period = least +ve int. N such that $C(D)$ divides $1 + D^N$.

[N is divisor of $2^L - 1$].

(ii) If $C(D)$ is a primitive poly^n. then each of the $2^L - 1$ non-zero initial states of non-singular LFSR $\langle L, C(D) \rangle$ produces an output seq. with maximum possible period $2^L - 1$.

* Barlekump Massey Algorithm:- To compute Linear Complexity profile.

• Def^n:- Consider the finite binary sequence

$$s^{N+1} = s_0, s_1, \dots, s_N$$

For $C(D) = 1 + C_1 D + \dots + C_L D^L$, let $\langle L, C(D) \rangle$ be an LFSR that generates the sequences $s^N = s_0, s_1, \dots, s_{N-1}$ The next discrepancy d_N is the difference betw^n s_N and the $(N+1)^{st}$ term generated by LFSR.

$$d_N = \left(s_N + \sum_{i=1}^{L} C_i s_{N-i} \right) \bmod 2.$$

• Fact:- Let $s^N = s_0, s_1, \dots, s_{N-1}$ be a finite binary seq. of linear complexity

$$L = L(s^N)$$ and let $\langle L, C(D) \rangle$ be a LFSR which generates s^N.

(i) LFSR $\langle L, C(D) \rangle$ also generates $s^{N+1} = s_0, s_1, \dots, s_{N-1}, s_N$ iff next discrepancy d_N equals 0.

(ii) If $d_N = 0$, then $L(s^{N+1}) = L$

(iii) If $d_N = 1$, let m be the largest integer $< N$ such that $L(s^m) < L(s^N)$ and let $\langle L(s^m), B(D) \rangle$ be an LFSR of length $L(s^m)$ which generates s^m. Then $\langle L', C'(D) \rangle$ is an LFSR of smallest length which generates s^{N+1},

, where

$$L' = \begin{cases} L & \text{if } L > N/2 \\ N+1-L & \text{if } L \leq N/2 \end{cases}$$

and $C'(D) = C(D) + B(D)\, D^{N-m}$

INPUT: A binary seq^n $s^n = s_0, s_1, \ldots, s_{n-1}$ of length n.

OUTPUT: The linear complexity $L(s^n)$ of s^n, $0 \leq L(s^n) \leq n$.

1.) Initialization: $C(D) \leftarrow 1$, $L \leftarrow 0$, $m \leftarrow -1$, $B(D) \leftarrow 1$, $N \leftarrow 0$.

2.) While $(N < n)$ do the following:

2.1.) Compute the next discrepancy d

$$d \leftarrow (s_N + \sum_{i=1}^{L} c_i s_{n-i}) \bmod 2.$$

2.2.) If $d = 1$, then do the following:

$T(D) \leftarrow C(D)$, $C(D) \leftarrow C(D) + B(D)\, D^{N-m}$

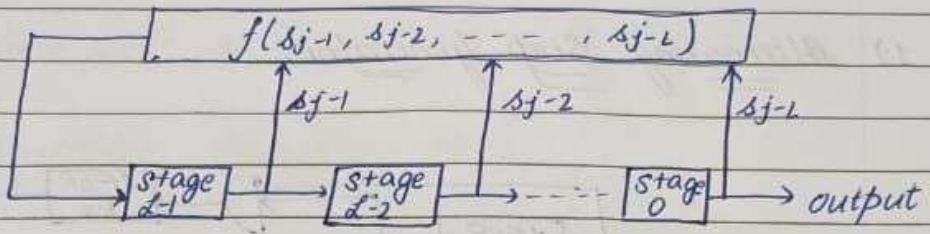
If $(L \leq N/2)$ then,

$L \leftarrow N+1-L$, $m \leftarrow N$, $B(D) \leftarrow T(D)$

2.3.) $N \leftarrow N+1$

3.) return (L).

* Non-Linear-FSR:-



$$s_j = f(s_{j-1}, s_{j-2}, \dots, s_{j-L}).$$

- **Def^n:-**

If the period of output sequence (for any initial steps) of a non-singular FSR of length L, is 2^L, then FSR is called a de Bruijn FSR and output sequence is called de Buijn sequence.

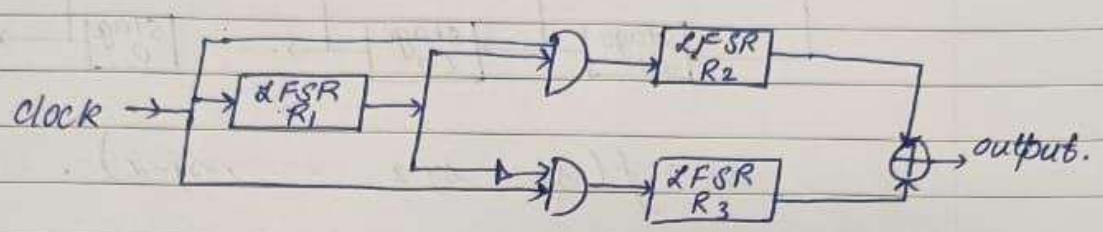
Example: $f(x_1, x_2, x_3) = 1 \oplus x_2 \oplus x_3 \oplus x_1 x_2$
for a 3-stage FSR.

t	stage 2	stage 1	stage 0
0	0	0	0
1	1	0	0
2	1	1	0
3	1	1	1
4	0	1	1
		⋮	

output seq. → 0,0,0,1,1,1,0,1.

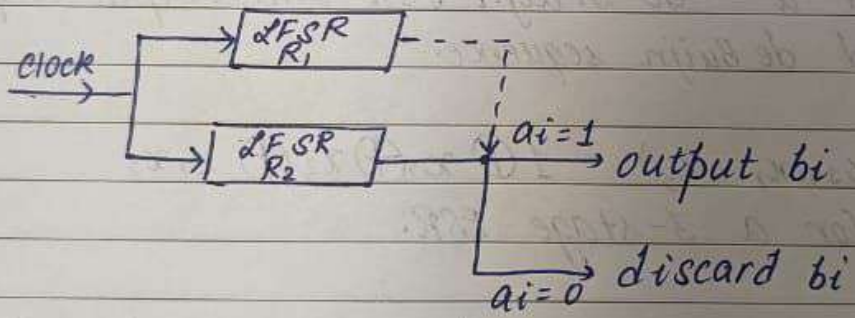
* Clock Controlled Generators:-

1.) Alternating Step generator:-



Period: $2^{L_1}(2^{L_2}-1)(2^{L_3}-1)$

2.) Shrinking generator:-



1.) Registers R_1 and R_2 are clocked.

2.) If R_1 is 1, the output bit of R_2 forms part of the keystream.

3.) If the output R_1 is 0, the output bit of R_2 is discarded.

- Fact:- If R_1 and R_2 are maximum LFSR L_1, L_2 resp. and

let x be the output seq. of shrinking generator then.

(i) If $\gcd(L_1, L_2) = 1$, then x has period $(2^{L_2}-1)(2^{L_1-1})$

★ One time pad:- ★

★ Cryptographic Hash fⁿs:- ★